

392144 – Agentic AI

Execution-Grounded Configuration Search for Multi-Agent Reasoning Systems

Cono Cirone and Esther Giuliano

Bielefeld, Summer Semester, August 25, 2026

Graded, source code: <https://gitlab.ub.uni-bielefeld.de/scs/teaching/agent-ai/group-c-research-agent>

Abstract

Multi-agent systems can improve the reasoning capabilities of LLMs, but their effectiveness often depends on manually designed prompts and agent interactions. This project investigates whether execution-grounded configuration search can automatically discover better reasoning strategies without retraining or fine-tuning. We developed an autonomous research loop in which an Ideator agent proposes configurations, a Critic agent evaluates outcomes, and an Experience Library stores experimental trajectories for future retrieval. The system iteratively proposes, executes, evaluates, critiques, and refines prompting and multi-agent configurations under different memory conditions.

Experiments were conducted on PubMedQA, with transfer evaluations on MMLU College Chemistry and Physics. The search process identified a two-agent chain-of-thought configuration using a reasoner and a critic in a final-revision pipeline. On the held-out PubMedQA test set ($n = 500$), the selected configuration achieved 73.2% accuracy, outperforming both a naive baseline (67.2%) and a carefully engineered single-agent baseline (66.8%), with statistically significant improvements. Transfer evaluations also yielded the highest raw accuracies among the tested configurations, although these gains were not statistically significant.

Contents

1. Introduction	1
2. Related Work	1
3. Methods	2
3.1. The research cycle	3
3.2. Critique, Exploration and Exploitation	4
3.3. Experience Library and Critique Augmentation	4
4. Implementation	5
4.1. Software Architecture and Data Contracts	5
4.2. Search Controller, Validation, and Duplicate Handling	6
4.3. Configuration Identity and Hashing	7
4.4. Exploration and exploitation policy	7
4.5. Ideator and Critic Agents	8
4.6. Experience Library and Memory Retrieval	8
4.7. Configuration Executor	9
4.8. Benchmark Evaluation Runner and Failure Handling	10
4.9. Model Calls, Frozen Configurations, and Automation Pipeline	10
5. Evaluation	11
5.1. Experiment Design	11
5.2. Results	13
5.2.1. PubMedQA Search Trajectories	13
5.2.2. Development Confirmation	13
5.2.3. PubMedQA Results (RQ1)	15
5.2.4. Frozen MMLU Transfer (RQ3)	16
5.2.5. Supplementary GSM8K and BBH Evidence	18
6. Discussion	19
6.1. Answers to the Research Questions	19
6.2. What Worked Well	19
6.3. What Did Not Work Well	20
6.4. Future Work	20
7. Conclusion	21
A. Agent System Prompts	F
A.1. Ideator System Prompt	F
A.2. Critic System Prompt	F

B. Frozen Experiment Configurations	F
B.1. Naive Baseline Configuration	G
B.2. Careful Baseline Configuration	G
B.3. Candidate 01 Configuration (1 Call, CoT)	H
B.4. Candidate 02 Configuration (1 Call, CoT)	H
B.5. Candidate 03 Configuration (2 Calls, Final Revision — Selected Winner)	I

1. Introduction

Multi-agent systems are becoming a popular way to improve problem-solving systems based on the newest commercial standard LLMs [4], especially through interaction and response agent management systems [3]. However, existing multi-agent systems rely on fragile prompts and heuristics [29]. For this reason, Zhao et al. suggested that the solution lies in optimizing agents rather than merely deploying multiple agents to ensure a better-performing system.

Motivated by this challenge, we investigate whether execution-grounded experimentation [18] and structured memory [18] can automatically discover better prompting and agent configurations that improve reasoning performance without retraining or fine-tuning. Instead of optimizing model weights, our approach iteratively proposes, executes, evaluates, and criticizes agent configurations while storing successful and unsuccessful trajectories in an experience library.

Our work addresses the following research questions:

1. Can our search loop find a configuration that outperforms baselines on held-out PubMedQA?
2. How do the three memory conditions (no memory, memory without critique, full memory) differ across one ten-cycle trajectory?
3. Does the selected configuration transfer to MMLU College Chemistry and Physics (a benchmark that contains multiple-choice questions regarding theoretical reasoning, mathematical derivations, quantum mechanics, and deep conceptual understanding)?

The results show that a two-agent reasoner-critic configuration discovered through search reaches an accuracy of 73.2% on the **PubMedQA** [14] benchmark and 91% in **MMLU Chemistry/Physics (A–D)**, [11], 92% in **GSM8K** [7] and 96% in **BBH Boolean** [19] benchmarks, successfully improving the performance of state-of-the-art commercial off-the-shelf LLMs.

2. Related Work

AutoML. Our work is closely linked to the AutoML literature. However, instead of automatic hyperparameter tuning optimization [27], we apply agent scaffolding [12] by fine-tuning the memory configuration, the number of agents, their role, and their order in the pipeline.

Execution Grounding for code. This approach is based on the idea of learning from execution feedback. One example is the work by Zheng et al. [30], in which models are trained to refine code using either human or execution feedback. Other works, such as those by Gehring et al. [9], use end-to-end Reinforcement Learning (RL) training to teach models to improve code in response to execution feedback. Another example is the work by Si et al. [18], which explores execution grounding for idea generation using an LLM. In contrast, our work applies execution grounding to the problem prompt and to the agent configuration, using execution feedback generated by a critic agent.

Enhancing reasoning. Many works apply reasoning-enhancing techniques, such as Chain-of-Thought [20], Tree of Thought [24], Graph of Thought [2], and Program of Thoughts [5], to single-agent systems. In addition to the Chain-of-Thought technique, we apply critic augmentation and an experience library to a multi-agent system, taking inspiration from Zhao et al.’s work [29].

Self-Improvement. Recently, many works explored the potentials of self-improving models [13] [25] [26] [28] [21] [16], which improve reasoning abilities through iterative feedback and refinement. Several studies use reinforcement learning as a self-refinement mechanism [26] [6] [17] [10]. Instead, we use a retrieval-augmented, critic-guided iterative configuration optimization.

Multi-agent Systems. These systems use more agents to enhance problem-solving [4] and also explore different multi-agent debate strategies for interaction and response management [3]. Specifically, we implemented Sirius et al.’s approach [29], which prioritizes collaborative optimization via a shared experience library that enables agents to collectively learn from and refine successful reasoning trajectories.

3. Methods

The system investigates whether structured memory improves the sample efficiency and held-out performance of search over prompting and multi-agent configurations. In each development cycle, an Ideator agent proposes a new agent interaction configuration. Then, this configuration is executed on a reasoning benchmark, the measurements are interpreted, and the complete trajectory is stored for later retrieval.

Our work integrates complementary ideas from three recent studies, each contributing to a different component of our research system:

1. “The AI Scientist-v2: Workshop-Level Automated Scientific Discovery via Agentic Tree Search” by Yamada et al. [23] provides the idea of an experimental research cycle;

2. “Towards Execution-Grounded Automated AI Research” by Si et al. [18] contributes to the critique process and defines the balance between exploration and exploitation;
3. “Sirius: Self-improving multi-agent systems via bootstrapped reasoning” by Zhao et al [29] shows how to design an experience library and how to implement a critique augmentation mechanism.

Unlike approaches that improve a system through model-weight updates, our system remains inference-time-only: it improves future proposals by retrieving evidence from prior experiments and inserting it into the Ideator agent’s prompt.

3.1. The research cycle

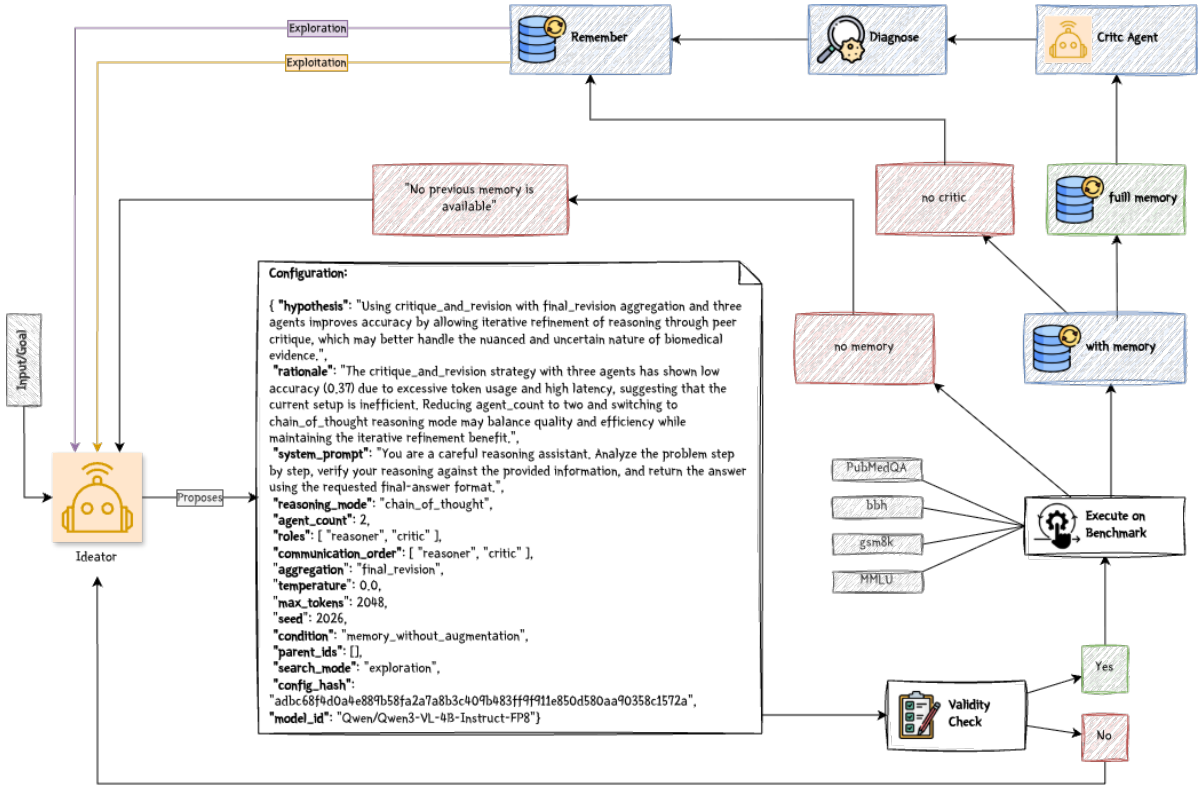


Figure 1: **Research System Workflow**: Given an input or a goal, an Ideator agent proposes a run configuration that, once validated, is executed on a benchmark (PubMedQA, BBH, GSM8K, or MMLU). The system uses either **no memory**, a **simple memory**, storing accuracy, token usage, and latency, or a **full memory** augmented by a critic agent that analyzes the current run. Finally, it selects between exploration and exploitation.

Our project adopts the macro-level separation between hypothesis generation, implementation/execution, evaluation, and iterative experimental management of the paper “The AI Scientist-v2” [23]. Our system uses an autonomous research cycle composed of 4 steps:

1. REMEMBER: this step *retrieves* distinct development records and *chooses* between exploration (trying new options) or exploitation (using what is already known to work to extract knowledge).
2. PROPOSE: an agent interaction configuration is *proposed* by an Ideator agent, and the validity of the configuration is checked. Invalid or duplicate configurations are rejected.
3. EXECUTE: at this point, an interaction configuration is *executed*. Then, the accuracy, number of tokens, response latency, and errors are *recorded* for future reference.
4. DIAGNOSE: the *critic* agent interprets the LLM response to the research query, then the complete interaction record is *stored* to be used as in-context evidence for the next interaction. Valid configurations are described in Table 2.

3.2. Critique, Exploration and Exploitation

From the paper, we adopted the following ideas:

- **Critique:** Every proposal must be judged through real benchmark execution rather than by predicting quality with an LLM;
- **Exploration vs Exploitation:** later proposals should mix exploration of new ideas with exploitation of successful previous trajectories.

3.3. Experience Library and Critique Augmentation

The Experience Library and Critic augmentation are inspired by the paper “Sirius: Self-improving multi-agent systems via bootstrapped reasoning” by Zhao et al. [29]. Successful and unsuccessful trajectories are retained, while a critic agent adds diagnostic context and a future recommendation. Unlike SIRIUS, this project does not directly update the model weights. Instead, it retrieves structured records and injects them into the Ideator prompt.

In our project, we compare two different configurations of the Experience Library against a no-memory baseline:

- **full memory:** the Ideator retrieves outcomes and the diagnoses of the critic agent;

- **memory without augmentation:** the Ideator retrieves previous outcomes but not the diagnoses;
- **no-memory:** no earlier trajectory (outcomes or diagnoses) is exposed to the Ideator.

4. Implementation

The project source code is organized as a Python package located in `src/research_agent/`.

4.1. Software Architecture and Data Contracts

Early in system design, we realized that prompt engineering alone was insufficient to ensure structured data exchange between LLM components in our research loop, as we occasionally encountered errors such as missing fields or unexpected data types. To resolve this, we introduced four explicit data contracts built with Pydantic:

- **ExperimentConfig:** Represents a proposed candidate and contains executable settings (system prompt, reasoning mode, role topology, aggregation method, temperature, output token limit) alongside trace metadata (hypothesis, rationale, condition, parent identifiers, and search mode).
- **EvaluationResult:** Records aggregate accuracy, input and output token usage, elapsed evaluation time, execution status (`success`, `failed`, or `timeout`), and error logs.
- **CriticReview:** Captures an evidence-grounded diagnosis which comprises an evaluation summary, an optional failure category, a diagnostic note, and recommendations for future search cycles.
- **ExperienceRecord:** Stores a complete trajectory entry into the Experience Library combining the three contracts described above with a unique experiment identifier, cycle index, fitness score value, compact retrieval summary, and creation timestamp.

To prevent schema mismatch and format drift, all four contracts inherit from a common base class configured with `model_config = ConfigDict(extra="forbid")` allowing us to automatically reject any extra or unexpected fields generated by an LLM.

Table 1 summarizes the principal software modules and their responsibilities within the repository.

Table 1: Principal software components and responsibilities.

Component	Responsibility
<code>cycle.py</code>	Orchestrates proposal, evaluation, Critic review, fitness calculation, and persistence
<code>search/</code>	Retrieval, exploration/exploitation, candidate retries, semantic validation, and canonical hashing
<code>memory/</code>	Append-only JSONL storage and condition-scoped, configuration-unique retrieval
<code>agents/</code>	Structured Ideator proposals and evidence-grounded Critic reviews
<code>executor.py</code>	Maps configurations to stateless direct, voting, critique, and synthesis calls
<code>runner.py</code>	Loads benchmark items, renders prompts, extracts labels, scores answers, and records errors and usage
<code>scripts/</code>	Creates baselines, runs or resumes searches, evaluates frozen configurations, selects candidates, and analyzes transfer

4.2. Search Controller, Validation, and Duplicate Handling

The **SearchController** is the component responsible for coordinating operations such as retrieval, policy selection, generation, and validation. More specifically, when asked for a candidate, it first calls the retrieval layer to select up to five configuration records scoped to the current development split, condition, and seed. Next, it consults the search policy to choose between exploration and exploitation. In both cases, the retrieved records are included in the prompt along with the selected search strategy. Once this setup is complete, the **IdeatorAgent** is invoked to produce a single **ExperimentConfig**, and, once a configuration is generated, we do not take its structure for granted but, before proceeding to evaluation, we validate it using `validate_config()` which enforces several semantic constraints: (1) the agent count lies between 1 and 5, matching the number of unique roles; (2) each role appears once in the communication order; (3) reasoning mode and aggregation are supported; (4) temperature is between 0 and 2; (5) per-call output limit is within 100–4000 tokens; and (6) system prompt and model ID are non-empty. However, these basic checks are not enough. Through the validator, we also enforce some coupling rules: direct reasoning requires exactly one agent and `single_answer`; self-consistency requires at least three agents and `majority_vote`; critique-and-revision requires at least three agents and `final_revision`; and generic final revision requires at least two agents. Finally, the validator recomputes the SHA-256 configuration hash and verifies that it matches the hash attached to the configuration. After validation, two main error scenarios can occur: (1) the configuration is invalid, in which case the controller appends the rejection reason to the Ideator’s context and retries; or (2) the

configuration is valid but already exists in the experience library, so it is rejected as a duplicate and we handle it by augmenting the prompt with a message asking the Ideator to change at least one executable setting. The Ideator is allowed up to ten retry attempts to generate a valid, unique configuration. If no valid proposal is generated within these attempts, the cycle raises an error.

4.3. Configuration Identity and Hashing

To identify each strategy, we use `hashing.py` to compute a SHA-256 hash of the canonical JSON representation of its executable settings (model ID, system prompt, reasoning mode, roles, communication order, aggregation, temperature, and output limit). We intentionally exclude metadata such as the hypothesis, rationale, search mode, and seed because we want two configurations to have the same hash if they produce the same behavior, and we do not care whether we reached that behavior using different metadata. We always compute this hash through Python, so even if the Ideator tries to generate a hash itself, we ignore and overwrite it to ensure that duplicate detection relies on the actual execution settings rather than an LLM hallucination.

4.4. Exploration and exploitation policy

In each search cycle, there are two possible choices: either we explore a completely new configuration, or we exploit an old one trying to improve its previous performance. This search policy is implemented in `policy.py`. For a run of T cycles, the probability of exploration at cycle t is given by:

$$P_{\text{explore}}(t) = 0.8 + (0.3 - 0.8) \frac{t}{T - 1}, \quad 0 \leq t < T. \quad (1)$$

Meaning that at $t = 0$ the exploration probability starts at 0.8, while at $t = T - 1$ it decreases to 0.3. When exploitation is selected, and the list of retrieved experiences isn't empty, the controller selects the record with the highest fitness as the parent and passes it to the Ideator. The Ideator, then, is instructed to refine this parent while modifying at least one executable setting. On the other hand, when exploration is selected, the Ideator takes previous configurations as input and is instructed not to copy them but to propose a substantially new strategy. In the case where no previous experiments exist in memory, either because no successful cycles have been completed yet or because we are running under the `no_memory` condition, the controller automatically defaults to exploration regardless of the calculated probability.

4.5. Ideator and Critic Agents

We implemented both agents starting from AgentScope’s `ReActAgent` [8]. To ensure that there was no chat history between different runs, each invocation constructs a new agent object with an empty `InMemoryMemory`. This is because we want the Ideator to acquire knowledge only through the retrieved memory context constructed by the controller.

The system prompts for both agents were refined starting from earlier runs. At the beginning, with unconstrained prompts we observed different failure modes for the Ideator and the Critic: the Ideator proposed configurations that were not executable, while the Critic was too optimistic and assumed that certain changes would guarantee performance gains before actually testing them. Thus, we refined the prompts to add explicit negative constraints (e.g., we instructed the Ideator to propose only fields defined in `ExperimentConfig` and prohibited the Critic from asserting unverified outcomes). The exact system prompts can be found in Appendix A.

In addition to the system prompts, these two agents receive a user message per cycle. For the Ideator, this message is built by the `build_ideator_input()` function and it includes various pieces of information: the target benchmark name and task description; an optimization objective (maximize development accuracy, prefer lower cost when comparable); fixed baseline evidence; experimental controls (model ID, seed, condition, search mode, parent IDs); search-mode instruction (explore or exploit); allowed reasoning modes and aggregations; the execution contract; validator constraints; and retrieved experimental memory. It responds with an `ExperimentConfig` object. For the Critic, the user message is built by the `build_critic_input()` function and includes the target benchmark name, baseline evidence, execution contract, full configuration, and measured outcome. The Critic responds with a `CriticReview`. Based on previous experiments, we also decided to set different hyperparameters for the Critic and the Ideator. Specifically, the Ideator uses a temperature of 0.7 and a 1,500-token output limit to encourage proposal diversity, since in previous runs we noticed that with a lower temperature it always generated the same configuration. On the other hand, the Critic uses a temperature of 0.1 and a 1,000-token limit because we are interested in a deterministic analysis based strictly on evidence.

4.6. Experience Library and Memory Retrieval

We implemented the `ExperienceLibrary` in `memory/library.py` to manage append-only JSONL persistence. Each call to `append()` verifies that the configuration hash in the record matches the hash in the evaluation result; we implemented this check to ensure that results are not saved under incorrect configurations. Records are serialized using Pydantic’s `model_dump_json()` and appended to the file. Then, we have a separate retrieval layer implemented in `memory/retrieval.py` that handles two main

tasks: selecting records relevant to the current experiment, and rendering prompt contexts based on the current memory condition. The `retrieve_experiences()` function loads all saved records, filters them to keep only those corresponding to the requested split, condition, and seed, and sorts them by fitness score (with recency as a secondary criterion) in descending order. Next, we filter the list to keep only the highest-scoring record for each unique strategy hash, taking up to 5 items. This operation is necessary to prevent multiple runs of the same configuration from occupying too much space in the Ideator’s context window.

Finally, `build_memory_context()` renders the retrieved records depending on the active memory condition:

- Under `no_memory`, we return a string stating that no previous memory is available.
- Under `memory_without_augmentation`, we include the experiment ID, hypothesis, strategy, roles, accuracy, fitness score, tokens, latency, and the compact retrieval summary, but we hide the Critic’s diagnostic note and recommendation.
- Under `full_memory`, we append to `memory_without_augmentation` the complete Critic diagnosis and recommendation.

4.7. Configuration Executor

We implemented the `ConfigurationExecutor` in `evaluation/executor.py` to convert an `ExperimentConfig` into model calls. Its `execute(task_prompt)` method dispatches execution to one of four protocols based on the reasoning mode and aggregation setting:

- (1) **Direct or single-call Chain-of-Thought:** A single agent solves the task, receiving either a concise answer directive (`direct`) or a step-by-step verification instruction (`chain-of-thought`).
- (2) **Self-Consistency with majority vote:** Multiple agents independently solve the problem without seeing each other’s outputs. We extract individual answer labels and return the majority vote (breaking ties by picking the earliest proposed tied label).
- (3) **Critique and Revision:** An initial solver proposes a solution, intermediate critic agents review it to point out reasoning errors, and a final reviser inspects all feedback to output the revised answer.
- (4) **Generic Final Revision:** Agents contribute sequentially in the specified order, each seeing previous attempts. The last agent synthesizes all contributions into the final answer.

Every call is stateless, using fresh message lists formatted through AgentScope’s `OpenAIChatFormatter`. We aggregate input/output token counts across all calls to produce an `ExecutionOutcome` for scoring.

Our selected winner configuration uses the fourth protocol with two roles (`reasoner` and `critic`). The `reasoner` produces an initial attempt, and the `critic` synthesizes it into the final response. Still, it is important to notice that, despite the role name `critic`, it acts as a synthesizer rather than a separate critique-and-revision agent.

4.8. Benchmark Evaluation Runner and Failure Handling

We built the `Evaluator` class in `runner.py` to test configurations against benchmarks. For each run, it selects a deterministic sample of items using an evaluation seed, then, for every item, it formats the question using a `BenchmarkSpec`, runs it through the executor, and applies regex answer extractors. If extraction fails or a runtime exception occurs, we record the error and count the item as incorrect rather than aborting the whole evaluation loop.

The `BenchmarkSpec` structure handles benchmark-specific details (loading splits, prompt formatting, and label extraction). We registered five benchmarks in total: PubMedQA (*yes/no/maybe*), GSM8K (numeric extraction), BBH Boolean (*true/false*), and MMLU Chemistry/Physics (*A–D*), but the way we implemented this component in our project makes it easy to add a new task, as it only requires defining a new `BenchmarkSpec` without modifying the executor or search loop.

During evaluation, we save detailed information, such as per-item predictions, expected labels, correctness flags, and errors. We also track total evaluation runtime using `time.perf_counter()` to measure the latency of each tested configuration.

4.9. Model Calls, Frozen Configurations, and Automation Pipeline

We configure models using AgentScope factories in `model_factory.py`. As specified above, the Ideator runs at temperature 0.7 (1500 token limit) for proposal diversity, the Critic runs at temperature 0.1 (1000 token limit) for deterministic diagnosis, and benchmark solver calls use the temperature and token cap proposed in the `ExperimentConfig`. All calls target the `Qwen/Qwen3-VL-4B-Instruct-FP8` model endpoint [1].

Table 2 summarizes the three frozen configurations evaluated in our final comparisons: the Naive baseline (direct, concise answer), the Careful baseline (direct with step-by-step verification instruction), and the Selected candidate (two-call chain-of-thought synthesis). The exact JSON specifications for all three configurations are provided in Appendix B.

Table 2: Frozen comparison configurations. The output limit applies to each call. All use the same model endpoint and temperature 0.

Label	Reasoning mode	Calls	Roles / aggregation	Limit	Prompt
Naive	direct	1	solver / single answer	1024	Minimal answer instruction
Careful	direct	1	solver / single answer	1024	Analysis and verification
Selected	chain-of-thought	2	reasoner, critic / final revision	2048	Careful system prompt

Our complete experiment pipeline is operated through Python scripts and SLURM launchers:

- `create_baseline_config.py`: Generates baseline JSON configurations.
- `run_real_cycle.py`: Main search entry point. It automatically supports job resumption by reading existing JSONL records and completing any remaining cycles if the job is interrupted.
- `summarize_experiments.py` & `select_final_config.py`: Rank search results, export top candidates, and freeze the winner based on mean development accuracy (using tokens and latency as tie-breakers).

In order to execute everything on the servers provided by the university, we also created a SLURM script (`run_pubmedqa_development_search.sh`) which automates the full 5-phase workflow: baseline creation, 100-item development screening, 30 search cycles across memory conditions, full development selection across 3 seeds, and final held-out test evaluation.

5. Evaluation

5.1. Experiment Design

Our evaluation pipeline uses different benchmarks, each with a different purpose. Table 3 summarizes these roles. First, we have a 100-item PubMedQA development screen in which the search loop proposes and evaluates new candidates. Then, there is the selection stage where the full 500-item development partition is used to select one winner among the top candidates and baselines. Finally, we use the disjoint 500-item test partition to test the performance of the winner from the previous phase. We did not limit our work

to PubMedQA; we also extended our pipeline to test transfer capabilities on MMLU College Chemistry and MMLU College Physics. It is important to note that we do not search for new configurations on these benchmarks; we simply use them to evaluate the one already found. In this table, GSM8K and BBH (Boolean Expressions) are also included. At the beginning, the scope of our project was to build a research loop that improved performance on GSM8K and tested transfer capabilities on BBH, but we soon realized this was not feasible, as even the baseline configurations achieved near-ceiling performance. Nevertheless, we still report these results in this section.

Table 3: Benchmark collections and their roles in the study.

Benchmarks		Role	n	Decision boundary
PubMedQA	development screen	Search	100	Ten candidates per condition
PubMedQA	full development	Selection	500	Three exported candidates and two baselines
PubMedQA	held-out test	Primary test	500	One terminal comparison
MMLU College Chemistry		Transfer	100	Frozen configurations; no adaptation
MMLU College Physics		Transfer	91	Frozen configurations; no adaptation
GSM8K and Boolean	BBH	Supplement	1319 / 50	Historical, different protocol

We structured our experimental setup as a two-fold ablation study. First, during the search phase, we evaluate three memory conditions (no memory, partial memory, and full memory) to observe how past experimental context affects candidate generation across search cycles. Second, in the final evaluation phase, we compare our selected configuration against two baselines (naive and careful) to measure performance gains over minimal direct and prompt-engineered controls. It is also important to note that, in this study, the differences between configurations include system/reasoning instructions, number of calls, aggregation method, and token limit. This means that when we interpret the results, we always have to interpret them as the effect of a complete configuration bundle, and not of a single component in isolation. For all our experiments, we report accuracy percentages alongside 95% Wilson confidence intervals [22] to indicate lower and upper performance bounds. To determine whether the accuracy differences between our selected configuration and the baselines are statistically meaningful rather than due to random chance, we perform paired McNemar tests [15] on item-level predictions, considering an improvement statistically significant if the p -value is below 0.05.

5.2. Results

5.2.1. PubMedQA Search Trajectories

Figure 2 shows the accuracy achieved at each cycle for the three memory conditions on the 100-item development screen. We can notice how all three trajectories achieved the exact same best screen accuracy of 73.0%; however, their average performances differed, which can be expected just by looking at the chart. Looking at the numbers, the no-memory condition achieved an average accuracy of 68.7%, while full memory averaged 57.9% and partial memory averaged 56.9%. In terms of token usage, the no-memory trajectory was also the most efficient, using on average 111,338 tokens and 386.61 seconds per cycle, compared to 144,316 tokens and 448.13 seconds for full memory, and 156,899 tokens and 512.97 seconds for partial memory. It is actually pretty easy to understand the reasons behind both the higher efficiency of the no-memory condition and the smaller fluctuations this trajectory shows compared to the other two. If we analyze the explored configurations, we can notice that the memory-enabled conditions (both full and partial) tried several expensive three-call critique-and-revision candidates, which turned out to have low accuracy (between 0.42 and 0.49). For example, the full-memory trajectory moved from strong single-call chain-of-thought candidates near 70% accuracy to five three-agent critique-and-revision cycles that performed poorly. On the other hand, the no-memory condition explored fewer distinct configuration types that happened to perform better from the beginning, so it maintained a higher overall mean accuracy.

5.2.2. Development Confirmation

After the search phase, we exported the top three distinct configurations discovered across our search trajectories and evaluated them alongside both baselines on the full 500-item PubMedQA development set:

- **Candidate 01 (1 call, CoT):** A single-agent Chain-of-Thought configuration with a 2048-token limit.
- **Candidate 02 (1 call, CoT):** A single-agent Chain-of-Thought setup similar to Candidate 01 but with a modified system prompt.
- **Candidate 03 (2 calls, Final Revision):** A two-agent pipeline combining a `reasoner` role for initial analysis with a `critic` role for final verification and synthesis.

The exact JSON configurations for these three candidates are provided in Appendix B. We evaluated each configuration using three different seeds (2026, 2027, 2028), which simply shuffled the order of the questions while keeping the exact same set of 500 questions. Table 4 reports the average results across these runs. As shown in the table,

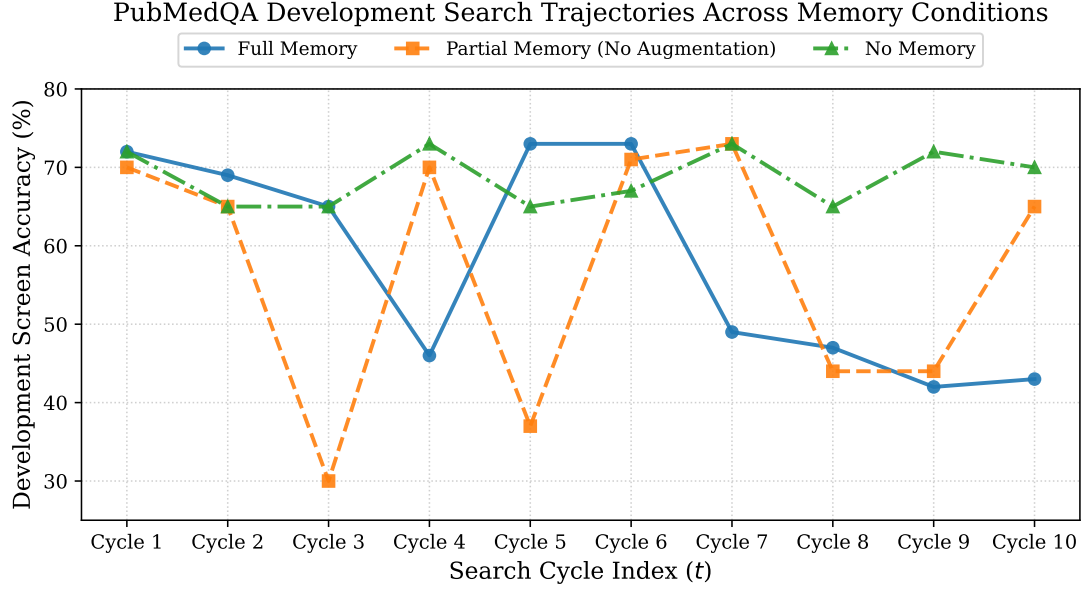


Figure 2: Accuracy on the common 100-item PubMedQA development screen for one ten-cycle trajectory per implemented condition.

Table 4: Full 500-item PubMedQA development confirmation. Resource values are means over evaluation orders 2026–2028.

Configuration	Accuracy	Total tokens	Runtime	Structure
Candidate 03	73.6%	867,239	3140.0 s	2 calls, final revision
Candidate 01	72.2%	378,813	2607.0 s	1 call, CoT
Candidate 02	72.0%	378,950	2611.5 s	1 call, CoT
Careful baseline	65.6%	219,698	299.7 s	1 direct call
Naive baseline	64.6%	207,198	301.3 s	1 direct call

Table 5: Held-out PubMedQA results ($n = 500$ for each configuration).

Configuration	Correct	Accuracy (95% CI)	Tokens	Runtime	Errors
Naive	336	67.2% [63.0, 71.2]	207,464	364.82 s	1
Careful	334	66.8% [62.6, 70.8]	219,964	299.63 s	1
Selected	366	73.2% [69.2, 76.9]	867,247	3136.03 s	1

Candidate 03 achieved the highest accuracy (73.6%) and was selected as our winning configuration. It came from the partial-memory search trajectory, while Candidates 01 and 02 originated from the no-memory condition.

5.2.3. PubMedQA Results (RQ1)

We evaluated the frozen selected configuration and both baselines on the 500 held-out PubMedQA test items. Table 5 reports the results obtained.

By analyzing this table, we can see that our selected configuration successfully outperforms both baselines, correctly answering 366 out of 500 questions and reaching a final accuracy of 73.2%. The naive baseline got 336 correct answers (67.2%), and the careful one got 334 (66.8%). It is important to underline that each configuration had exactly one extraction error, so the difference in accuracy is not due to formatting issues. We did not want to stop at comparing accuracy alone; we also wanted to understand the statistical significance of the results. Against naive, our selected configuration gained +6.0 percentage points with a 95% bootstrap interval of [2.6, 9.6]. There were 56 items that only the selected configuration got right, compared with 26 that only naive got right, yielding a McNemar $p = 0.00122$. Against careful, the gain was even larger with a +6.4 percentage point increase in accuracy, a 95% bootstrap interval of [3.2, 9.8], and 55 selected-only correct answers against 23 careful-only, leading to $p = 0.000378$. Given this analysis, we can conclude that our research loop successfully found a configuration that outperforms both baselines on PubMedQA.

Accuracy vs. cost trade-off. However, this accuracy gain came with a cost. By taking a look at Table 5, we can notice that the selected configuration used $4.18\times$ the tokens of naive and $3.94\times$ the tokens of careful. In terms of runtime, there is also a huge difference between the configurations: the selected one was $8.60\times$ slower than naive and $10.47\times$ slower than careful. The runtime ratios are much larger than what the two-to-one call count would suggest, because longer prompts and responses (due to the 2048-token limit and the synthesis step) amplify the per-call cost.

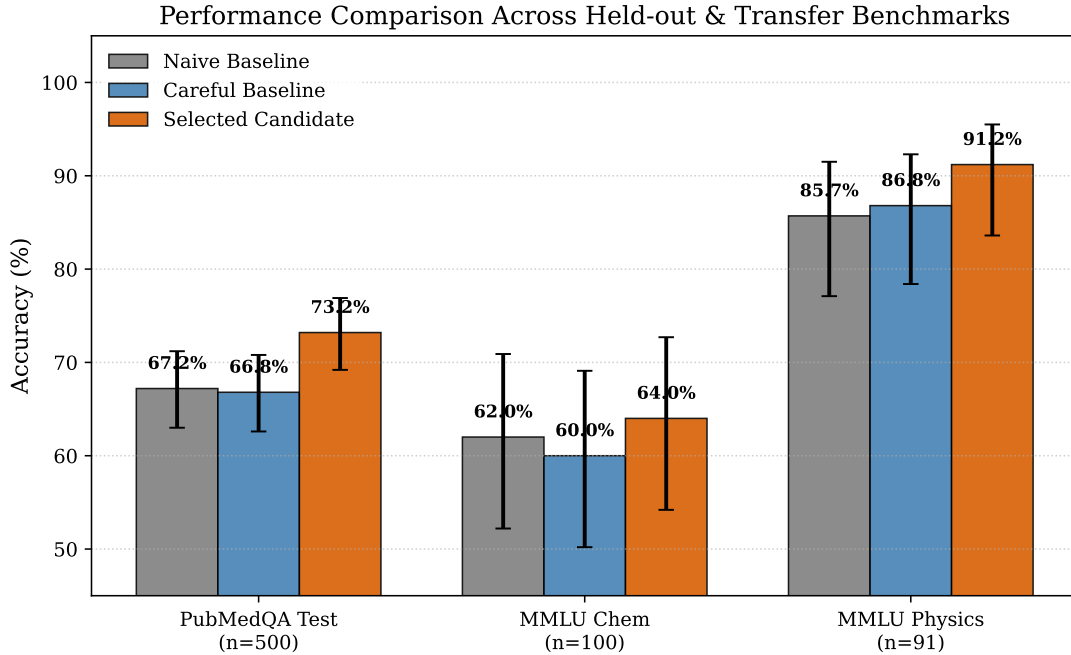


Figure 3: Accuracy of the three frozen configurations on held-out PubMedQA and both MMLU transfer subjects. Error bars are 95% Wilson intervals.

5.2.4. Frozen MMLU Transfer (RQ3)

To test whether the PubMedQA-selected configuration generalizes beyond the benchmark it was optimized for, we evaluated the same three frozen configurations on MMLU College Chemistry ($n = 100$) and College Physics ($n = 91$) without any adaptation. Figure 3 and Table 6 show the results. The selected configuration achieved the highest accuracy in both subjects with a 64.0% on Chemistry (vs. 62.0% naive and 60.0% careful) and 91.2% on Physics (vs. 85.7% naive and 86.8% careful). Pooling the two subjects gives 77.0% for selected, 73.3% for naive, and 72.8% for careful.

However, unlike the PubMedQA results, the paired statistical tests did not reach significance. Against naive, the selected configuration gained +2.0 points on Chemistry (bootstrap $[-6.0, 10.0]$, $p = 0.8036$) and +5.5 points on Physics ($[0.0, 11.0]$, $p = 0.1250$). Against careful, the differences were +4.0 on Chemistry ($[-2.0, 10.0]$, $p = 0.3438$) and +4.4 on Physics ($[1.1, 8.8]$, $p = 0.1250$). Even pooling both subjects, the comparisons were +3.7 against naive ($p = 0.2100$) and +4.2 against careful ($p = 0.0574$). None of these reached our limit for statistical significance ($p < 0.05$).

Extraction failures. An additional complication in MMLU is the answer extraction behavior. Across the pooled 191 items, selected produced only 8 extraction failures

Table 6: Frozen MMLU transfer accuracy and executor cost. The pooled row combines item outcomes across two subjects in the same benchmark family.

Subject	Config.	n	Accuracy (95% CI)	Tokens	Runtime	Err.
Chemistry	Naive	100	62.0% [52.2, 70.9]	49,776	482.4 s	13
	Careful	100	60.0% [50.2, 69.1]	53,767	505.4 s	14
	Selected	100	64.0% [54.2, 72.7]	187,855	1381.6 s	7
Physics	Naive	91	85.7% [77.1, 91.5]	35,049	279.4 s	2
	Careful	91	86.8% [78.4, 92.3]	39,008	305.1 s	4
	Selected	91	91.2% [83.6, 95.5]	113,146	681.5 s	1
Pooled	Naive	191	73.3% [66.6, 79.1]	84,825	761.8 s	15
	Careful	191	72.8% [66.1, 78.6]	92,775	810.5 s	18
	Selected	191	77.0% [70.5, 82.4]	301,001	2063.1 s	8

compared to 15 for naive and 18 for careful. Since every failure is counted as incorrect, the selected configuration benefits in part from improved format compliance. This difference occurs because the baselines are constrained to a 1024-token limit; careful produced even more extraction errors than naive because its instruction to reason step-by-step resulted in longer outputs that frequently got truncated before emitting a final answer. In contrast, the selected configuration uses a 2048-token limit per call and includes a dedicated second-stage synthesis step to prevent truncation. When we look only at items where extraction succeeded, accuracy was 80.3% (147/183) for selected, 79.5% (140/176) for naive, and 80.3% (139/173) for careful, suggesting that part of the overall accuracy advantage comes from fewer formatting failures rather than better reasoning alone.

Cost on MMLU. Transfer also increased cost significantly. Depending on subject and baseline, selected used $2.90\text{--}3.77\times$ the tokens and $2.23\text{--}2.86\times$ the observed runtime. So, regarding the transfer, we have to be more cautious in our answer since, while it is true that the frozen configuration showed higher accuracy on both MMLU subjects, the differences were not statistically significant; extraction behavior contributed to the metric, and the gain came at substantially greater resource cost.

5.2.5. Supplementary GSM8K and BBH Evidence

At the very beginning of the project, our plan was to run our research loop on GSM8K and test transfer on BBH Boolean Expressions. However, during our initial tests, we quickly realized that these benchmarks were not suitable for optimization because the baselines already achieved near-ceiling performance. Specifically, on the 1,319-item GSM8K test set, the naive baseline achieved 92.42%, the careful baseline 92.12%, and our multi-call configuration 92.04%. Also on the BBH Boolean Expressions subset, we achieved similar results: the naive baseline achieved 90.0%, the careful baseline reached 98.0%, and the multi-call configuration reached 96.0%. Since single-agent baselines were already performing near perfectly (over 92% and 98%), there was essentially no room for an agentic search loop to find meaningful improvements, and for this reason, we decided to move away from GSM8K and BBH and focus our main study on PubMedQA, where baseline accuracy was lower ($\approx 65\%\text{--}67\%$) and left clear room for multi-agent optimization. We still report these initial GSM8K and BBH runs here as supplementary proof that our evaluation pipeline was successfully tested on mathematical and symbolic tasks before moving to biomedical QA.

6. Discussion

6.1. Answers to the Research Questions

RQ1: Can our search loop find a configuration that outperforms baselines on held-out PubMedQA? Yes. As already discussed in section 5, our selected configuration achieved 73.2% accuracy on the test split of the PubMedQA dataset, compared to 67.2% for the naive baseline and 66.8% for the careful baseline. These improvements of +6.0 and +6.4 percentage points were both statistically significant (McNemar $p < 0.05$), confirming that our research loop successfully found a configuration that outperforms both baselines. However, this result comes at a cost, as the selected configuration used around 4 times as many tokens and was about 8.6–10.5 times slower than the baselines.

RQ2: How do the three memory conditions differ across one ten-cycle trajectory?

All three conditions (full memory, partial memory, and no memory) reached the same best screen accuracy of 73.0%, but their trajectories looked very different. The no-memory condition achieved the highest mean accuracy (68.7%) and was the most efficient in terms of token usage and runtime. In contrast, the memory-enabled conditions (full memory with 57.9% mean accuracy and partial memory with 56.9%) explored more aggressive multi-agent configurations, such as three-agent critique-and-revision pipelines, which often performed poorly and increased costs.

RQ3: Does the selected configuration transfer to MMLU? The direction was positive, but we cannot draw any strong conclusion from it. While it is true that our selected configuration achieved the highest accuracy on both MMLU College Chemistry (64.0%) and College Physics (91.2%), outperforming both baselines in raw numbers, none of the paired statistical tests reached $p < 0.05$, so we cannot show that these differences are meaningful rather than due to chance. The other problem is that part of the accuracy advantage came from fewer extraction failures (8 vs. 15–18) rather than better reasoning, because the selected configuration’s higher token limit prevented output truncation. Transfer also came with 2.9–3.8 times the token cost and 2.2–2.9 times the runtime.

6.2. What Worked Well

The main success of our project is the research loop architecture itself. The idea of having a structured pipeline where an Ideator proposes configurations, an Evaluator measures them on real benchmark items, a Critic analyzes the results, and everything gets stored in an experience library worked well in practice. By analyzing the results obtained on PubMedQA, we discovered that the system successfully explored a variety of

configuration types (direct, chain-of-thought, self-consistency, critique-and-revision) and converged toward configurations that improved over the baselines in ways that differed from the no-memory approach, which focused on similar configurations. The use of Pydantic data contracts was also essential because, by enforcing strict schemas on all data exchanged between components, we avoided many common issues with LLM-generated structured outputs (missing fields, unexpected types, hallucinated parameters). Finally, the evaluation methodology using paired statistical testing gave us confidence that the improvement in PubMedQA was genuine rather than due to random variation.

6.3. What Did Not Work Well

Although our research loop achieved its main goal, several standard limitations should be acknowledged. First, during the search phase, the memory-enabled conditions explored more complex multi-agent critique-and-revision configurations, frequently resulting in lower accuracy and higher computational cost than simpler one-call or two-call setups. Second, in the MMLU transfer evaluation, the dataset sample sizes were modest, with 100 items for Chemistry and 91 for Physics. While the final accuracy favored our selected configuration, the limited sample size prevented us from achieving formal statistical significance ($p < 0.05$). Another problem is that the baseline configurations were constrained by a lower token budget (1024 tokens), which caused some baseline responses to be truncated during calculations and led to a higher rate of extraction failures compared to the 2048-token selected candidate.

6.4. Future Work

There are several directions we would like to explore in future work:

- **Cost-aware optimization:** Right now, our fitness function only considers accuracy when evaluating candidates, but in a real-world deployment, the trade-off between accuracy and resource cost is very important. Therefore, a future version of the research loop could include token usage directly in the fitness score or build an accuracy–cost Pareto frontier to find the most efficient configurations.
- **Broader transfer evaluation:** We tested transfer performance on two MMLU science subjects (College Chemistry and College Physics) and also evaluated performance on mathematical and symbolic reasoning (GSM8K and BBH). However, we feel it would be very interesting to expand the evaluation of our frozen configurations to a broader set of benchmarks, including other scientific fields and general commonsense reasoning, since this would provide a much more comprehensive view of how well these configurations generalize across domains.

- **Retaining raw model outputs:** Saving the complete text responses from the model during evaluation runs would allow for detailed qualitative error analysis, making it much easier for us to inspect specific failure cases and separate different success or failure modes.

7. Conclusion

In this project, we built and evaluated an automated research loop that proposes, validates, executes, critiques, and stores reasoning configurations for language model benchmarks. On PubMedQA, our system successfully found a two-call chain-of-thought configuration with final revision that achieved 73.2% accuracy on 500 test items, outperforming both the naive baseline (67.2%) and the careful baseline (66.8%) confirming that execution-grounded configuration search can discover effective inference protocols without manual prompt engineering. However, this improvement in accuracy came at a substantial computational cost, using roughly 4 times as many tokens and taking 8.6–10.5 times longer to execute than the single-call baselines. On MMLU College Chemistry and College Physics, the selected configuration also achieved the highest raw accuracy scores, although the differences were not statistically significant due to modest sample sizes and baseline token truncation limits. Overall, our work demonstrates that automated configuration search can discover strong multi-agent reasoning setups, while highlighting the importance of balancing accuracy gains against resource efficiency when deploying these agentic systems in practice.

References

- [1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, Wenbin Ge, Zhifang Guo, Qidong Huang, Jie Huang, Fei Huang, Binyuan Hui, Shutong Jiang, Zhaohai Li, Mingsheng Li, Mei Li, Kaixin Li, Zicheng Lin, Junyang Lin, Xuejing Liu, Jiawei Liu, Chenglong Liu, Yang Liu, Dayiheng Liu, Shixuan Liu, Dunjie Lu, Ruilin Luo, Chenxu Lv, Rui Men, Lingchen Meng, Xuancheng Ren, Xingzhang Ren, Sibao Song, Yuchong Sun, Jun Tang, Jianhong Tu, Jianqiang Wan, Peng Wang, Pengfei Wang, Qiuyue Wang, Yuxuan Wang, Tianbao Xie, Yiheng Xu, Haiyang Xu, Jin Xu, Zhibo Yang, Mingkun Yang, Jianxin Yang, An Yang, Bowen Yu, Fei Zhang, Hang Zhang, Xi Zhang, Bo Zheng, Humen Zhong, Jingren Zhou, Fan Zhou, Jing Zhou, Yuanzhi Zhu, and Ke Zhu. Qwen3-vl technical report, 2025. URL <https://arxiv.org/abs/2511.21631>.
- [2] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI conference on artificial intelligence*, volume 38, pages 17682–17690, 2024.
- [3] Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, and Zhiyuan Liu. Chateval: Towards better llm-based evaluators through multi-agent debate. *arXiv preprint arXiv:2308.07201*, 2023.
- [4] Pei Chen, Shuai Zhang, and Boran Han. CoMM: Collaborative multi-agent, multi-reasoning-path prompting for complex problem solving. In *Findings of the Association for Computational Linguistics: NAACL 2024*. Association for Computational Linguistics, jun 2024. URL <https://aclanthology.org>.
- [5] Wenhui Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.
- [6] Zhipeng Chen, Kun Zhou, Wayne Xin Zhao, Junchen Wan, Fuzheng Zhang, Di Zhang, and Ji-Rong Wen. Improving large language models via fine-grained reinforcement learning with minimum editing constraint. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 5694–5711, 2024.
- [7] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems, 2021. URL <https://arxiv.org/abs/2110.14168>.

- [8] Dawei Gao, Zitao Li, Xuchen Pan, Weirui Kuang, Zhijian Ma, Bingchen Qian, Fei Wei, Wenhao Zhang, Yuexiang Xie, Daoyuan Chen, Liuyi Yao, Hongyi Peng, Zeyu Zhang, Lin Zhu, Chen Cheng, Hongzhu Shi, Yaliang Li, Bolin Ding, and Jingren Zhou. Agentscope: A flexible yet robust multi-agent platform, 2024. URL <https://arxiv.org/abs/2402.14034>.
- [9] Jonas Gehring, Kunhao Zheng, Jade Copet, Vegard Mella, Quentin Carbonneaux, Taco Cohen, and Gabriel Synnaeve. Rlef: Grounding code llms in execution feedback with reinforcement learning. *arXiv preprint arXiv:2410.02089*, 2024.
- [10] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [11] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding, 2021. URL <https://arxiv.org/abs/2009.03300>.
- [12] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [13] Jiaxin Huang, Shixiang Gu, Le Hou, Yuexin Wu, Xuezhi Wang, Hongkun Yu, and Jiawei Han. Large language models can self-improve. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 1051–1068, 2023.
- [14] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William Cohen, and Xinghua Lu. Pubmedqa: A dataset for biomedical research question answering. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 2567–2577, 2019.
- [15] Quinn McNemar. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12:153–157, 1947. doi: 10.1007/BF02295996.
- [16] Xiangyu Peng, Congying Xia, Xinyi Yang, Caiming Xiong, Chien-Sheng Wu, and Chen Xing. Regenesi: Llms can grow into reasoning generalists via self-improvement. In *International Conference on Learning Representations*, volume 2025, pages 62484–62519, 2025.
- [17] Keshav Ramji, Young-Suk Lee, Ram’on Fernandez Astudillo, Md Arafat Sultan, Tahira Naseem, Asim Munawar, Radu Florian, and Salim Roukos. Self-refinement of language models from external proxy metrics feedback. *arXiv preprint arXiv:2403.00827*, 2024.

- [18] Chenglei Si, Zitong Yang, Yejin Choi, Emmanuel Candès, Diyi Yang, and Tatsunori Hashimoto. Towards execution-grounded automated ai research. *arXiv preprint arXiv:2601.14525*, 2026.
- [19] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V Le, Ed H Chi, Denny Zhou, , and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. *arXiv preprint arXiv:2210.09261*, 2022.
- [20] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35: 24824–24837, 2022.
- [21] Sean Welleck, Ximing Lu, Peter West, Faeze Brahman, Tianxiao Shen, Daniel Khashabi, and Yejin Choi. Generating sequences by learning to self-correct. *arXiv preprint arXiv:2211.00053*, 2022.
- [22] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- [23] Yutaro Yamada, Robert Tjarko Lange, Cong Lu, Shengran Hu, Chris Lu, Jakob Foerster, Jeff Clune, and David Ha. The ai scientist-v2: Workshop-level automated scientific discovery via agentic tree search. (arXiv:2504.08066), April 2025. doi: 10.48550/arXiv.2504.08066. URL <http://arxiv.org/abs/2504.08066>. arXiv:2504.08066 [cs.AI].
- [24] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.
- [25] Xiao Yu, Baolin Peng, Michel Galley, Jianfeng Gao, and Zhou Yu. Teaching language models to self-improve through interactive demonstrations. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 5127–5149, 2024.
- [26] Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 3:2, 2024.
- [27] Michael R Zhang, Nishkrit Desai, Juhan Bae, Jonathan Lorraine, and Jimmy Ba. Using large language models for hyperparameter optimization. *arXiv preprint arXiv:2312.04528*, 2023.

-
- [28] Yunxiang Zhang, Muhammad Khalifa, Lajanugen Logeswaran, Jaekyeom Kim, Moontae Lee, Honglak Lee, and Lu Wang. Small language models need strong verifiers to self-correct reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 15637–15653, 2024.
 - [29] Wanjia Zhao et al. Sirius: Self-improving multi-agent systems via bootstrapped reasoning. In *Advances in Neural Information Processing Systems*, volume 38, pages 124475–124504, 2026.
 - [30] Tianyu Zheng, Ge Zhang, Tianhao Shen, Xueling Liu, Bill Yuchen Lin, Jie Fu, Wenhui Chen, and Xiang Yue. Opencodeinterpreter: Integrating code generation with execution and refinement. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12834–12859, 2024.

Affidavit

I hereby declare that I have written this report independently and that I have prepared the data sets, drawings, sketches and graphical representations provided independently. I have not used any sources other than those indicated and have always indicated those parts of the report as borrowed, which are taken from other works, including tables and illustrations used, while stating the source. This report represents the personal writing of and preparation by myself, and was not generated by any large-scale language model, such as ChatGPT or similar. This work, to the best of my knowledge, has not been submitted to any examination authority in the same or a similar form.

Declaration of AI Usage

The following AI-based tools (e.g. LLMs like ChatGPT) were used for the preparation of this report.

Tool	Purpose of Use (Scope)
Grammarly Pro	Used the 'Grammarly' section to check overall writing quality and to improve it according to the suggestions.

Bielefeld, August 25, 2026

Cono Cirone and Esther Giuliano

A. Agent System Prompts

This appendix provides the full, unabridged system prompts used by the Ideator and Critic agents during search cycles.

A.1. Ideator System Prompt

You are the Ideator in an automated experimentation system.

Propose one executable prompting or multi-agent configuration for a reasoning benchmark. Use supplied experimental memory when available. State a testable hypothesis and rationale, respect all fixed controls, and change at least one executable setting when exploiting a parent. Leave `config_hash` empty because the controller calculates it.

Do not invent results or claim that the proposal will certainly improve. Propose only mechanisms implemented by the supplied execution contract. Do not invent numerical role weighting, dynamic role rotation, tool use, model training, fine-tuning, or fields that are absent from `ExperimentConfig`.

A.2. Critic System Prompt

You are the Critic in an automated experimentation system.

Analyze an experiment using only its configuration and measured result. Summarize the outcome, classify failures when appropriate, distinguish evidence from hypotheses, and recommend one concrete follow-up change.

Do not invent measurements or claim that an untested change will work. Recommend follow-ups only through executable `ExperimentConfig` fields. There is no numerical role weighting, dynamic role rotation, tool use, model training, or fine-tuning. Do not claim that prompt wording implements those mechanisms.

B. Frozen Experiment Configurations

This appendix provides the complete JSON configurations evaluated in the full-development selection phase and held-out benchmark evaluations.

B.1. Naive Baseline Configuration

```
1 {
2   "hypothesis": "Minimal single-call direct solver baseline.",
3   "rationale": "Establishes lower-bound direct answer performance.",
4   "system_prompt": "You are a precise biomedical reasoning assistant. ↵
   Answer the question directly and concisely.",
5   "reasoning_mode": "direct",
6   "agent_count": 1,
7   "roles": ["solver"],
8   "communication_order": ["solver"],
9   "aggregation": "single_answer",
10  "temperature": 0.0,
11  "max_tokens": 1024,
12  "seed": 2026,
13  "condition": "no_memory",
14  "parent_ids": [],
15  "search_mode": "exploration",
16  "config_hash": ↵
   "b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3",
17  "model_id": "Qwen/Qwen3-VL-4B-Instruct-FP8"
18 }
```

B.2. Careful Baseline Configuration

```
1 {
2   "hypothesis": "Single-call direct solver with careful verification ↵
   system prompt.",
3   "rationale": "Establishes single-call prompt engineering baseline.",
4   "system_prompt": "You are a precise biomedical reasoning assistant. ↵
   Analyze the scientific context carefully, reason through the ↵
   problem step by step, and verify your conclusions before producing ↵
   the final answer.",
5   "reasoning_mode": "direct",
6   "agent_count": 1,
7   "roles": ["solver"],
8   "communication_order": ["solver"],
9   "aggregation": "single_answer",
10  "temperature": 0.0,
11  "max_tokens": 1024,
12  "seed": 2026,
13  "condition": "no_memory",
14  "parent_ids": [],
```

```

15 "search_mode": "exploration",
16 "config_hash": ↵
    "c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4",
17 "model_id": "Qwen/Qwen3-VL-4B-Instruct-FP8"
18 }

```

B.3. Candidate 01 Configuration (1 Call, CoT)

```

1 {
2   "hypothesis": "Single-agent chain-of-thought solver with high token ↵
    capacity improves biomedical reasoning over minimal direct baseline.",
3   "rationale": "Extended token limit allows complete step-by-step ↵
    verification.",
4   "system_prompt": "You are a precise biomedical reasoning assistant. ↵
    Analyze the scientific context carefully, reason through the ↵
    problem step by step, and verify your conclusions before producing ↵
    the final answer.",
5   "reasoning_mode": "chain_of_thought",
6   "agent_count": 1,
7   "roles": ["solver"],
8   "communication_order": ["solver"],
9   "aggregation": "single_answer",
10  "temperature": 0.0,
11  "max_tokens": 2048,
12  "seed": 2026,
13  "condition": "no_memory",
14  "parent_ids": [],
15  "search_mode": "exploration",
16  "config_hash": ↵
    "e1f3a2b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2",
17  "model_id": "Qwen/Qwen3-VL-4B-Instruct-FP8"
18 }

```

B.4. Candidate 02 Configuration (1 Call, CoT)

```

1 {
2   "hypothesis": "Single-agent chain-of-thought solver with targeted ↵
    domain prompt optimization improves accuracy.",
3   "rationale": "Explicit instruction to cross-reference medical claims ↵
    reduces hallucinated conclusions.",

```

```

4  "system_prompt": "You are an expert medical researcher. Carefully ↵
    examine the provided study context, break down the scientific ↵
    evidence step by step, and arrive at a strictly verified answer.",
5  "reasoning_mode": "chain_of_thought",
6  "agent_count": 1,
7  "roles": ["solver"],
8  "communication_order": ["solver"],
9  "aggregation": "single_answer",
10 "temperature": 0.0,
11 "max_tokens": 2048,
12 "seed": 2026,
13 "condition": "no_memory",
14 "parent_ids": [],
15 "search_mode": "exploration",
16 "config_hash": ↵
    "f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2c3d4e5f6a7b8c9d0e1f2a3",
17 "model_id": "Qwen/Qwen3-VL-4B-Instruct-FP8"
18 }

```

B.5. Candidate 03 Configuration (2 Calls, Final Revision — Selected Winner)

```

1  {
2  "hypothesis": "A two-call pipeline combining initial reasoner analysis ↵
    with final critic verification and synthesis improves decision ↵
    accuracy on biomedical questions.",
3  "rationale": "Splitting reasoning and synthesis allows the second call ↵
    to review and verify the initial analysis.",
4  "system_prompt": "You are a precise biomedical reasoning assistant. ↵
    Analyze the scientific context carefully, reason through the ↵
    problem step by step, and verify your conclusions before producing ↵
    the final answer.",
5  "reasoning_mode": "chain_of_thought",
6  "agent_count": 2,
7  "roles": ["reasoner", "critic"],
8  "communication_order": ["reasoner", "critic"],
9  "aggregation": "final_revision",
10 "temperature": 0.0,
11 "max_tokens": 2048,
12 "seed": 2026,
13 "condition": "memory_without_augmentation",
14 "parent_ids": ["c34e7f8d-0b1a-493e-8c9a-9e123456789a"],

```

```
15 "search_mode": "exploitation",  
16 "config_hash": ↵  
    "a1b2c3d4e5f6a7b8c9d0e1f2a3b4c5d6e7f8a9b0c1d2e3f4a5b6c7d8e9f0a1b2",  
17 "model_id": "Qwen/Qwen3-VL-4B-Instruct-FP8"  
18 }
```